"For everything there is a season… a time to seek, and a time to lose." — Ecclesiastes 3:1,6

Unsupervised Learning: K-Means Clustering

Course Instructor: Prasanna Ranjith Christodoss
Associate Professor of Comp. Sc., Messiah University, PA 17055

Miroslav Kubat, ***An Introduction to Machine Learning***
Tan, Steinbach & Kumar,
Introduction to Data Mining

Lecture 11

Lecture Outline

1. What is Cluster Analysis?
2. Representing Clusters by Centroids
3. Measuring Distance (Euclidean)
4. Which Cluster Should an Example Belong To?
5. Types of Clustering
6. The K-Means Algorithm (4 Steps)
7. Convergence & Termination
8. Worked Example (K=2)
9. Initialization Strategies
10. Python Demo: K-Means on Iris Dataset

Key Concept

K-Means groups unlabelled data into K clusters by minimising the total distance of each point to its cluster centroid — with NO class labels required.

What is Cluster Analysis?

Definition

Finding groups of objects such that objects ***within a group are similar*** to one another but ***different*** from objects in other groups.

Input: A set of examples described by attribute vectors — **NO class labels.**

Output:
Two or more non-overlapping clusters where each example belongs to exactly one cluster.

Within Cluster

Examples are close to each other (low intra-cluster distance).

Between Clusters

Examples are far from other clusters (high inter-cluster distance).

Non-Overlapping

Each example belongs to one and only one cluster (hard assignment).

Representing Clusters by Centroids

A centroid is the average position of all points in a cluster — the "centre of gravity."

📍 Location

The centroid marks where a cluster is positioned in feature space. Clusters are typically summarised by their centroids.

∑ Calculation

For numeric attributes, average each attribute independently across all examples in the cluster.

Example: Points (2,5),(1,4),(3,6) → centroid = (2, 5)

🔢 Discrete Attributes

Convert discrete attributes to Boolean (one-hot) encoding, then compute averages normally.

E.g. season=fall → [1,0,0], season=winter → [0,1,0]

Centroid formula: $c = (1/n) \times \Sigma x_i$ where n = number of points in the cluster

Measuring Distance: Euclidean

$$d(p, q) = \sqrt{(p_1-q_1)^2 + (p_2-q_2)^2 + \ldots + (p_n-q_n)^2}$$

1D (line)

$d = |p_1 - q_1|$

p=(3), q=(7) → d = 4

2D (plane)

$d = \sqrt{(p_1-q_1)^2+(p_2-q_2)^2}$

p=(1,2), q=(4,6) → d = 5

nD (general)

$d = \sqrt{\Sigma_i (p_i-q_i)^2}$

Used for any number of features in ML

Cluster Assignment Rule

Assign example x to cluster i* where: $i^* = \text{argmin}\ d(x, c_i)$ — the cluster whose centroid is closest.

Types of Clustering

Partitional (Unnested)

Divides data into **non-overlapping** subsets.
Each example belongs to exactly ONE cluster.

Key algorithm: K-Means

Strengths:
- Simple and efficient
- Scales well to large datasets
- Easy to interpret

Limitation:
- Must specify K in advance
- Sensitive to outliers

Hierarchical (Nested)

Creates a **tree of nested clusters** (a dendrogram).
Any level of the tree gives a valid grouping.

Key algorithms: Agglomerative, Divisive

Strengths:
- No need to pre-specify K
- Visual insight via dendrogram
- Detects sub-clusters

Limitation:
- Expensive for large datasets
- Hard to undo merges once made

The K-Means Algorithm

1

Initialize

Choose K initial centroids — either randomly from data points, or using domain knowledge. Create K initial clusters by assigning each example to its nearest centroid.

2

Assign

For each example x, compute $d(x, c_i)$ for all K centroids. Assign x to the cluster with the smallest distance: cluster = argmin $d(x, c_i)$.

3

Update Centroids

Recalculate each centroid as the mean of all examples now assigned to that cluster: $c_i = (1/|C_i|) \times \Sigma\, x$ for $x \in C_i$.

4

Repeat Until Convergence

Go back to step 2. Stop when no example changes cluster during a full pass (an "epoch") — the assignment is stable.

Objective: minimise $J = \Sigma_i\ \Sigma_{x \in C_i}\ ||x - c_i||^2$ (total within-cluster sum of squares)

Convergence & Termination

Guaranteed Convergence

K-Means always converges because inertia (J) strictly decreases (or stays the same) at every iteration. It cannot increase — so the process must eventually stabilise.

Non-Overlapping Clusters

Each example is assigned to exactly one cluster at every step — clusters are always disjoint (no shared members).

Termination Condition

Stop when NO example is relocated during a complete epoch.
Equivalently: cluster assignments are identical to the previous iteration.

Important Caveats

Local minimum:
K-Means converges to a LOCAL optimum, not necessarily the global best. Different initializations can produce different results.

Must specify K:
K must be chosen before running the algorithm. Use the Elbow Method or Silhouette Score to find a good K.

Spherical clusters:
K-Means assumes clusters are roughly spherical and of similar size — it struggles with elongated or irregular shapes.

Outlier sensitivity:
Outliers can pull centroids away from the true cluster centres (use K-Medoids for robustness).

Worked Example (K = 2)

Dataset

i	X	Y
A	1	1
B	1	0
C	0	2
D	2	4
E	3	5

Iteration 0 — Initialise

```
Choose K=2. Select A(1,1) and C(0,2) as initial centroids.
C1=(1,1),  C2=(0,2)
```

Iteration 1 — Assign & Update

```
Compute distances → assign:
  A,B → Cluster 1   |   C,D,E → Cluster 2
New centroids: C1=(1,0.5),  C2=(1.67,3.67)
```

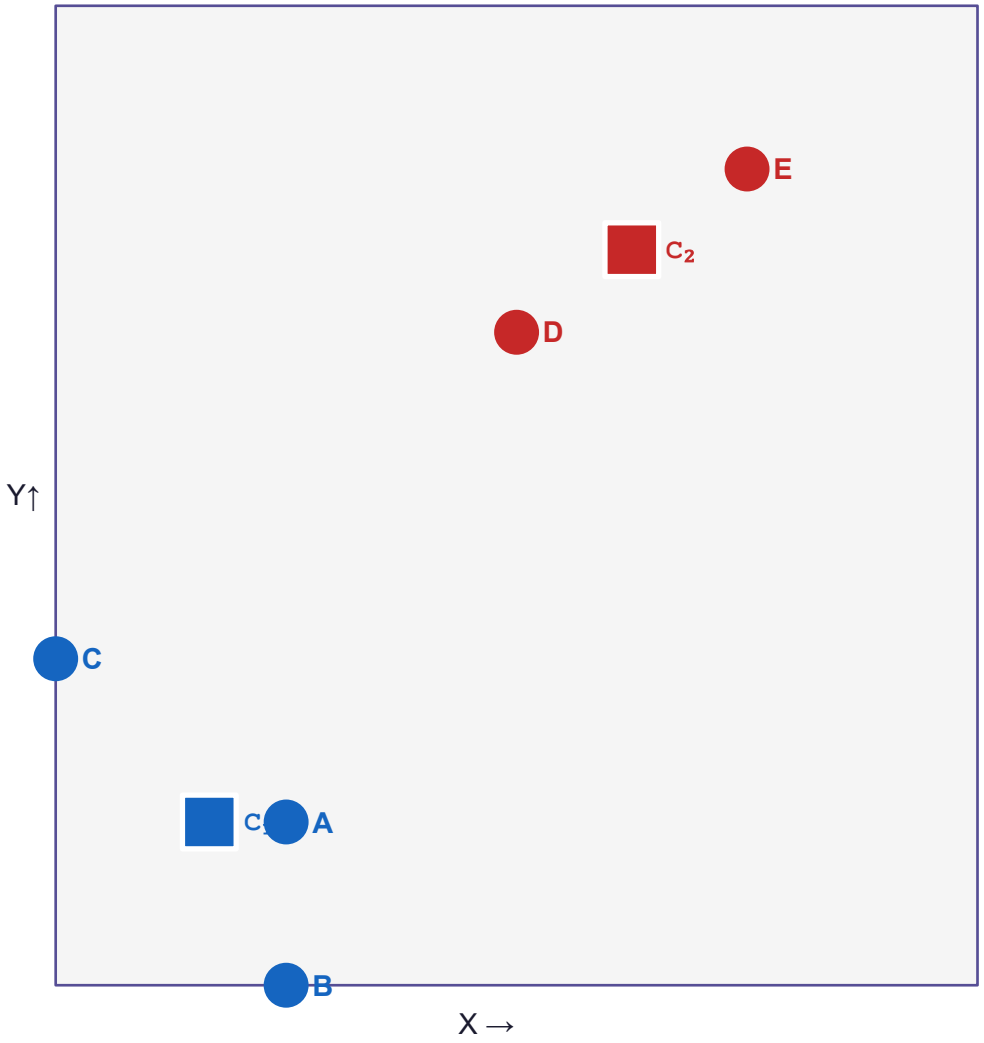
Iteration 2 — Reassign & Update

```
Recompute distances → C moves to Cluster 1:
  A,B,C → Cluster 1   |   D,E → Cluster 2
New centroids: C1=(0.67,1.0),  C2=(2.5,4.5)
```

Iteration 3 — Converged ✓

```
No reassignments. Final clusters stable.
  Cluster 1: {A,B,C}   Centroid=(0.67,1.0)
  Cluster 2: {D,E}     Centroid=(2.5,4.5)
```

Final Clusters Visualised (K = 2)



Final State

Cluster 1 (Blue)

Points: A(1,1), B(1,0), C(0,2)
Centroid: (0.67, 1.0)

Cluster 2 (Red)

Points: D(2,4), E(3,5)
Centroid: (2.5, 4.5)

- ■ *Squares = centroids*
- ● *Circles = data points*

No further reassignments → algorithm converged in 3 iterations.

Initialization Strategies

The starting centroids heavily influence the final clusters. Good initialization → fewer iterations, better solution.

1

Domain Knowledge

Use known groupings from the problem domain as starting clusters.

Example: segment employees by age groups (20–35, 36–50, 51+) to initialise salary/salary clusters.

Best when: prior knowledge about structure exists.

2

Random Initialisation

Randomly select K examples from the dataset as initial centroids.

Simple and fast but may yield poor local minima.

Fix: run K-Means multiple times with different random seeds → keep the run with lowest inertia.

3

K-Means++ (Recommended)

Choose first centroid randomly. Each subsequent centroid is chosen with probability proportional to its squared distance from the nearest existing centroid.

Spreads initial centroids out → typically converges to better solutions with fewer iterations.

Default in scikit-learn.

Python: K-Means on Iris Dataset

Python

```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# -- Load data (2 features for visualisation) ----------
iris = load_iris()
X = iris.data[:, :2]          # sepal length & width
X_scaled = StandardScaler().fit_transform(X)

# -- Elbow method: find optimal K ---------------------
inertias = []
for k in range(1, 9):
    km = KMeans(n_clusters=k, random_state=42, n_init='auto')
    km.fit(X_scaled)
    inertias.append(km.inertia_)

# -- Fit final model with K=3 -------------------------
km3 = KMeans(n_clusters=3, init='k-means++',
             random_state=42, n_init='auto')
km3.fit(X_scaled)
labels  = km3.labels_
centers = km3.cluster_centers_

# -- Plot clusters ------------------------------------
colors = ['#1565C0','#C62828','#2E7D32']
for k in range(3):
    mask = labels == k
    plt.scatter(X_scaled[mask,0], X_scaled[mask,1],
                c=colors[k], label=f'Cluster {k+1}', alpha=0.7)
plt.scatter(centers[:,0], centers[:,1],
            c='gold', marker='X', s=200,
            edgecolors='black', label='Centroids')
plt.title('K-Means Clustering — Iris (K=3)')
plt.xlabel('Sepal Length (scaled)')
plt.ylabel('Sepal Width (scaled)')
plt.legend(); plt.tight_layout(); plt.show()
```

Load & Scale

Load Iris (150 samples, 4 features). Use first 2 for plotting. StandardScaler brings features to same scale — essential for distance-based algorithms.

Elbow Method

Run K-Means for K=1…8. Plot inertia vs K. Look for the "elbow" where gain slows — that K is optimal.

K-Means++ Fit

Use K=3 (matching the 3 Iris species). init="k-means++" gives smarter initialisation for better results.

Visualise

Scatter plot colour-coded by cluster. Gold ✕ markers show centroids. Compare with true species labels to evaluate.

Summary

1

K-Means is Unsupervised

No class labels are required. The algorithm discovers natural groupings in unlabelled data.

2

Centroid-Based Assignment

Each example is assigned to the cluster whose centroid is nearest (Euclidean distance). Centroids are the average of all cluster members.

3

4-Step Iterative Loop

Initialize → Assign → Update Centroids → Repeat. Terminates when no reassignments occur (convergence).

4

Initialization Matters

Random init can hit poor local minima. K-Means++ (default in sklearn) spreads seeds for faster, better convergence.

5

Find K with the Elbow Method — Plot inertia vs K. The "elbow" point is where adding more clusters gives diminishing returns on inertia reduction.